CM1 : Tests Unitaires avec pytest

Configuration · Fixtures · Paramétrisation · Mocking Redis

Plan du cours

01 Configuration de pytest

pytest.ini · conftest.py · Fixtures SQLite

02 Écriture des tests

Structure · Classes · Assertions · pytest.raises

03 Paramétrisation avancée

@pytest.mark.parametrize · Cas limites · Markers

Mocking Redis

mocker.patch · assert_called · MagicMock

P A R T I E 0 1

Configuration de pytest

pytest.ini · conftest.py · Fixtures SQLite :memory:

Pourquoi pytest ?

Découverte auto

pytest trouve tous les fichiers test_*.py et fonctions test_*
sans configuration manuelle

Syntaxe simple

Un simple assert suffit
Pas besoin de self.assertEqual() comme avec unittest

Fixtures puissantes

Les fixtures remplacent setUp/tearDown avec yield et scope
automatique

Écosystème riche

pytest-cov, pytest-mock, pytest-xdist, pytest-asyncio
Tous intégrés à ShopFlow

Le fichier pytest.ini : Configuration globale

Placé à la racine du projet, il configure le comportement de pytest pour toute la suite de tests.

```
# pytest.ini — racine de shopflow/
[pytest]
addopts = -v --cov=app --cov-report=html --cov-report=term-
missing
#          ↑ verbose   ↑ mesurer le coverage   ↑ rapport HTML   ↑
lignes manquantes
testpaths = tests           # dossier de recherche des tests
python_files = test_*.py    # fichiers reconnus comme tests
python_classes = Test*      # classes reconnues
python_functions = test_*   # fonctions reconnues

markers =                   # markers personnalisés
    unit: tests unitaires isolés
    integration: tests API + BDD
    slow: tests > 1s
```

→ -v : affiche le nom de chaque test

→ --cov=app : mesure app/ uniquement

→ --cov-report=html : génère htmlcov/

→ testpaths : évite de chercher partout

→ markers : filtre pytest -m unit

Le fichier conftest.py — Fixtures partagées

conftest.py est détecté automatiquement par pytest
Les fixtures qu'il définit sont disponibles dans TOUS les fichiers de test sans import.

```
# tests/conftest.py
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool
from app.database import Base
from app.models import Product, Coupon

@pytest.fixture(scope='module')
def db_engine():
    engine = create_engine('sqlite:///:memory:',
        connect_args={'check_same_thread': False},
        poolclass=StaticPool)
    Base.metadata.create_all(engine)
    yield engine
    Base.metadata.drop_all(engine)  # teardown
```

scope='module'

créée UNE FOIS par fichier de test

sqlite:///:memory:

BDD en RAM , rapide et isolée

StaticPool

connexion unique partagée (obligatoire)

create_all()

crée les tables Product, Cart, Order...

yield

code avant = setup, après = teardown

Les 4 scopes de fixtures pytest

scope= 'function'

Recréée pour CHAQUE test (défaut)
Détruite après chaque test

Ex : db_session → isolation totale

```
test_1 → création fixture → test → destruction
test_2 → création fixture → test → destruction
test_3 → création fixture → test → destruction
```

scope= 'class'

Partagée entre les tests d'une classe

Ex : état commun à TestCalculPrixTtc

```
ClasseTest
   ├─ test_1
   ├─ test_2
   └─ test_3
```

```
création fixture
   ├─ test_1
   ├─ test_2
   └─ test_3
destruction fixture
```

scope= 'module'

Partagée entre tous les tests d'un fichier

Ex : db_engine, client TestClient

```
test_file.py
   ├─ test_a
   ├─ test_b
   └─ test_c
```

```
création fixture
   ├─ test_a
   ├─ test_b
   └─ test_c
destruction fixture
```

scope= package'

Créée une fois par dossier de tests

```
tests/
   ├─ test_a.py
   ├─ test_b.py
   └─ test_c.py
```

```
création fixture
   ├─ test_a.py
   ├─ test_b.py
   └─ test_c.py
destruction fixture
```

scope= 'session'

Partagée pour TOUTE la suite de tests

```
pytest run
   ├─ test_a
   ├─ test_b
   ├─ test_c
   └─ ...
```

```
création fixture
   ├─ tous les tests
destruction fixture à la fin
```

P A R T I E 0 2

Écriture des tests

Structure · Classes · Assertions · pytest.raises

Anatomie d'un test pytest : Structure AAA

Chaque test suit le pattern AAA : Arrange (préparer) → Act (agir) → Assert (vérifier).

```python
# tests/unit/test_pricing.py
import pytest
from app.services.pricing import calcul_prix_ttc
from app.models import Coupon

@pytest.mark.unit              # ← marker personnalisé
class TestCalculPrixTtc:       # ← regroupement logique

    def test_prix_normal(self):    # ← nom explicite
        # ARRANGE — préparer les données
        prix_ht = 100.0

        # ACT — appeler la fonction à tester
        result = calcul_prix_ttc(prix_ht)

        # ASSERT — vérifier le résultat
        assert result == 120.0  # 100 × 1.20 = 120
```

ARRANGE — Préparer les données

ACT — Appeler la fonction

ASSERT — Vérifier le résultat

Règle : 1 test = 1 assertion principale

Tester les exceptions avec pytest.raises

pytest.raises() capture l'exception attendue. Sans lui, une exception non gérée fait ÉCHOUER le test.

```
# CORRECT — vérifier type + message
def test_prix_negatif_leve_exception(self):
    with pytest.raises(ValueError, match='invalide'):
        calcul_prix_ttc(-5.0)
    # ↑ match= vérifie que le MESSAGE contient 'invalide'

# MOINS PRÉCIS — vérifie seulement le type
def test_moins_precis(self):
    with pytest.raises(ValueError):  # n'importe quelle
ValueError
        calcul_prix_ttc(-5.0)

# MAUVAIS — le test passe même si aucune exception
def test_mauvais(self):
    try:
        calcul_prix_ttc(-5.0)
    except ValueError:
        pass   # ← ne vérifie RIEN
```

match= — Pourquoi ?

- Vérifie le TYPE de l'exception
- ET que le MESSAGE contient la chaîne
- Protège contre les faux positifs
- Ex: ValueError('mauvaise raison') serait rejeté

Attributs accessibles

- excinfo.type → type de l'exception
- excinfo.value → l'objet exception
- excinfo.traceback → la pile d'appel

P A R T I E 0 3

Paramétrisation avancée

@pytest.mark.parametrize · Cas limites · Markers

@pytest.mark.parametrize — Tester plusieurs cas

Sans paramétrisation, on copie le même test 4 fois. Avec parametrize, pytest génère N tests distincts automatiquement.

```
# SANS parametrize code dupliqué
def test_10_pourcent(): assert appliquer_coupon(100, ...)
== 90
def test_50_pourcent(): assert appliquer_coupon(200, ...)
== 100
def test_100_pourcent(): assert appliquer_coupon(50, ...)
== 0

# AVEC parametrize   DRY (Don't Repeat Yourself)
@pytest.mark.parametrize('reduction,prix,attendu', [
    (10,  100.0,  90.0),   # -10%
    (50,  200.0, 100.0),   # -50%
    (100,  50.0,   0.0),   # gratuit
    (1,   100.0,  99.0),   # minimal
])
def test_reductions(reduction, prix, attendu):  # pytest
crée 4 tests
    coupon = Coupon(code=f'T{reduction}',
reduction=float(reduction), actif=True)
    assert appliquer_coupon(prix, coupon) == attendu
```

4 tests générés

Noms : test_reductions[10-100.0-90.0]

Échec ciblé

Si 50% échoue, les 3 autres passent

Lisible

Le tableau = la spécification du comportement

DRY

Un seul assert pour tous les cas

Les Markers pytest : Filtrer et organiser les tests

```
# 1. Déclarer les markers dans pytest.ini
[pytest]
markers =
    unit: tests unitaires sans BDD ni réseau
    integration: tests API avec BDD
    slow: tests > 1 seconde
    pricing: tests du module pricing

# 2. Appliquer sur une classe ou une fonction
@pytest.mark.unit
class TestCalculPrixTtc:
    @pytest.mark.slow
    def test_calcul_lent(self): ...

# 3. Filtrer à l'exécution
# pytest -m unit          → seulement les tests unitaires
# pytest -m 'not slow'    → exclure les tests lents
# pytest -m 'unit and pricing' → intersection
```

ShopFlow : pytest -m unit → 30 tests en 0.3s | pytest (tout) → 65 tests en 2s

Cas d'usage en CI/CD

- Pipeline rapide : pytest -m 'not slow'
- Pipeline complet : pytest (tout)
- Nuit : pytest -m slow
- Débogage : pytest -m pricing -v

Markers non déclarés

- PytestUnknownMarkWarning si marker
- non déclaré dans pytest.ini
- Toujours déclarer dans [pytest] markers

P A R T I E 0 4

Mocking Redis

mocker.patch · assert_called · MagicMock

Pourquoi mocker Redis ?

Sans mock

```
def reserver_stock(product, qty, session):
    product.stock -= qty
    session.commit()
    redis_client.delete(...)  # ← VRAI Redis

# Redis non démarré →
# ConnectionRefusedError !
```

Avec mock

```
mock_redis = mocker.patch(
    'app.services.stock.redis_client')

reserver_stock(product, 3, session)
# Redis remplacé par MagicMock
# Aucun appel réseau réel
# Test isolé et rapide
```

Règle fondamentale du test unitaire :

Mocker TOUTES les dépendances I/O : Redis, base de données distante, API externe, emails, fichiers.

mocker.patch : La règle du chemin

Règle critique : patcher là où l'objet est UTILISÉ, pas là où il est DÉFINI.

```
# Dans app/services/stock.py :
from app.cache import redis_client   # importé ici

def reserver_stock(product, qty, session):
    ...
    redis_client.delete(...)   # utilisé ici

# MAUVAIS mocker là où il est défini
mocker.patch('app.cache.redis_client')  # ne fonctionne
PAS

# CORRECT mocker là où il est utilisé
mocker.patch('app.services.stock.redis_client')
#             ↑ module qui UTILISE redis_client
```

Pourquoi ?

Quand stock.py fait :
from app.cache import redis_client

Python COPIE la référence
dans app.services.stock

Mocker app.cache.redis_client
modifie l'original, mais stock.py
garde sa COPIE locale → le mock
n'a aucun effet !

Il faut remplacer la COPIE
dans app.services.stock

Vérifier les appels avec assert_called_*

```
mock_redis = mocker.patch('app.services.stock.redis_client')

reserver_stock(product, 3, db_session)

# Vérifier que delete() a été appelé
mock_redis.delete.assert_called_once()          # 1 fois exactement
mock_redis.delete.assert_called_once_with(key)  # 1 fois avec cet arg
mock_redis.delete.assert_called()               # au moins 1 fois
mock_redis.delete.assert_not_called()           # jamais appelé

# Inspecter les arguments
args = mock_redis.delete.call_args              # derniers arguments
calls = mock_redis.delete.call_args_list        # tous les appels
count = mock_redis.delete.call_count            # nombre d'appels
```

MagicMock : L'objet silencieux universel

mocker.patch() remplace l'objet par un MagicMock
Il accepte TOUS les appels sans erreur.

```
from unittest.mock import MagicMock

mock = MagicMock()

# Accepte n'importe quel appel
mock.get('product:1:stock')      # → MagicMock()
mock.set('k', 'v')              # → MagicMock()
mock.delete('k')                # → MagicMock()
mock.nimporte_quoi.foo.bar()    # → MagicMock()

# Configurer des valeurs de retour
mock.get.return_value = '42'          # get() retourne '42'
mock.ping.side_effect = Exception()   # ping() lève une
exception

# Dans app/cache.py fallback si Redis absent
return MagicMock()    # développement sans Docker
```

Utilisé dans ShopFlow

- cache.py : MagicMock si Redis absent
- test_stock.py : mocker.patch remplace redis_client par un MagicMock
- Permet de dev sans Docker
- assert_called vérifie les effets de bord

Ne pas abuser

- Mocker la logique interne = mauvais
- Mocker uniquement les I/O : Redis,
- SMTP, APIs externes, fichiers

Récapitulatif : Ce qu'il faut retenir

01

pytest.ini + conftest.py

- pytest.ini : addopts, testpaths, markers
- conftest.py : fixtures partagées automatiquement
- scope=module pour db_engine (partagé par fichier)
- scope=function pour db_session (isolé par test)

02

Écriture des tests

- Pattern AAA : Arrange → Act → Assert
- 1 test = 1 comportement à vérifier
- pytest.raises(ValueError, match='msg')
- Classes Test* pour regrouper logiquement

03

Paramétrisation

- @pytest.mark.parametrize génère N tests
- Tableau de cas = spécification exécutable
- pytest -m unit pour filtrer
- Déclarer markers dans pytest.ini

04

Mocking Redis

- mocker.patch('MODULE_UTILISATEUR.redis_client')
- PAS 'app.cache.redis_client' (module définisseur)
- assert_called_once() / assert_not_called()
- MagicMock accepte tous les appels